



KNOWING YOUR BOOK WITH INKHAVEN

Know Your Book

The Intelligence That Reads Along With You

Vladimir Ulogov

2026 · examples assume Inkhaven 2.6.0 or newer

Contents

What this book covers	1
Who this book is for	2
One promise about cost	3
Part I — The Ground Truth	4
1 — The Facts Beneath	5
The bible your book checks itself against	6
The fact-check — is the page true to the bible?	6
When the facts disagree with each other	7
2 — The Knowledge Graph	9
What the edges know	10
Ask your book a question	10
The graph builds itself	11
Part II — The Book Watches Itself	13
3 — Continuity — Where and When	14
One ledger, every detector	15
The dashboard, and the jump	15
The subtle cases, on request	16
4 — Who Knows What, When	17
The grant — when could they know it?	18
The use — and the break	18
Where it lives	19
Part III — The Book Reads Itself	21
5 — The Read-Through	22
Shape — measured from the prose	23
Audience — what a first reader trips on	23
A reader's eyes, on request	24
6 — The Voices	25
A fingerprint, and a distinctiveness matrix	26
Whose head, and does the tense hold	26
The Inner Stylist	27
Part IV — Knowing You're Getting Somewhere	28
7 — Did It Get Better?	29
Mark, then trend	30
Cleared, and introduced	30
Part V — All Together	32

8 — A Scene Through Every Check	33
First, is it true to the world?	34
Then, does the continuity hold?	34
Could she know what she confronts Sella with?	34
Do they sound like two people, and how does it read?	35
Act on it — and confirm it landed	35
Appendix A — The Knowledge Commands	37
From the command line	37
Inside the editor	38
The tags that declare knowledge (KEN)	39
From a script (Bund)	39
Appendix B — Glossary	40
About the Author	44
Why these intelligences exist	45
A work of love	45
A note on cooperation	46
Where to find more	46

Before You Begin

There is a moment, somewhere past the halfway mark of a long manuscript, when the book grows larger than the head holding it. You can no longer remember whether the war ended in the spring or the autumn, whether your detective has met the widow yet, whether chapter nine still drags the way it did three revisions ago. The book knows these things. You wrote them. But knowing and **holding** are different, and no writer holds a whole book at once.

This is a book about the second kind of knowing. Inkhaven cannot write your novel — and would not try — but it can **hold your book for you**: remember its facts, watch its continuity, track who knows what, read it the way a stranger would, listen to its voices, and tell you, draft to draft, whether it is getting better. Every one of these is a way of **knowing your book**, and this guide is a tour of all of them.

What this book covers

Inkhaven's knowledge features are not one tool but a family, each answering a different question you would otherwise have to answer from memory:

Is this fact right, and where did it come from? the
Facts bible

How does everything in my book connect? the
knowledge graph

Is my continuity holding — place, time, numbers?
SENTINEL

Could this character know this yet? KEN

How does the whole thing read, start to end?
LECTOR

Do my characters sound like different people?
CHORUS

Did this revision actually make it better?
CHRONICLE

You do not have to use all of them, and few books need every one. Read the chapter for the question that is nagging you today; the book is arranged so each stands on its own.

Who this book is for

This guide assumes **no prior knowledge** — of Inkhaven, or of the craft-analysis ideas behind these features. Every term is introduced the first time it is needed, in a marked box:

TERM **Advisory**

Every feature in this book is **advisory**. It observes, measures, and reports; it never edits your prose on its own. The reader points; the author decides. When a change is made, you see it as a diff and confirm it, and your old words are always one keystroke back.

It also keeps two kinds of author in view — the novelist and the non-fiction writer — because the intelligence serves both, though it serves them differently:

FICTION

The novelist leans hardest on **continuity**, **who-knows-what**, and **voice** — the machinery of a world with secrets and a cast who must each sound like themselves.

NON-FICTION

The non-fiction writer leans hardest on the **facts bible** and the **graph** — the machinery of claims that must be right, sourced, and coherent with each other.

One promise about cost

The word “intelligence” makes writers reach for their wallet, expecting a meter to spin every time the tool thinks. It does not. Almost everything in this book is **deterministic and free** — it reads your manuscript and its own structure, with no model involved. Where a language model genuinely helps — the subtle continuity break, a synthetic first read — it is always **explicit and cost-capped**: you ask for it, you see the estimate, and the default is off. Cost **informs**; it never surprises.

So: turn the page, and let us begin with the ground the whole book stands on — the facts it did not invent.

WHAT YOU LEARNED

- Inkhaven’s **knowledge features** are a family of tools that **hold your book for you** — its facts, continuity, who-knows-what, how it reads, how it sounds, and whether it is improving.
- Every feature is **advisory** — it observes and reports, and never edits your prose without a diff you confirm.
- The core is **deterministic and free**; any language-model help is **explicit and cost-capped**, never automatic.

KNOW YOUR BOOK

PART I

The Ground Truth

CHAPTER 1

1

The Facts Beneath

Every book rests on things that must stay true. In a novel it is the world's own law — the queen's name, the distance between two cities, the year the plague came. In non-fiction it is the borrowed fact — the population, the date, the finding a reader could look up. Get one wrong once and a reader shrugs; get it wrong **twice, differently**, and the trust is gone. Inkhaven gives these must-stay-true things a home of their own: the **Facts** book.

TERM Fact

A **fact** is a specific, checkable claim your book rests on — “the war ended in 412,” “Vienna sat on the Danube,” “she is the elder sister.” It lives in the Facts system book, apart from your prose, so the tool can hold it steady while you write around it.

The bible your book checks itself against

The Facts book is your story bible made **operational**. It is not a document you read; it is a set of claims the tool can test your manuscript against. You add to it as you go — a name here, a date there — and Inkhaven keeps them organised the way a book is, in foldable sections.

● ● ● *The Facts book — the world's must-stay-true claims*

- ▼ Facts
 - ▼ People
 - ✓ Mara is the elder of the two sisters
 - ✓ Joren served in the northern war
 - ▼ Chronology
 - ✓ The war ended in the year 412
 - ✓ The coronation follows the war by three years
 - ▼ Geography
 - ✓ Rillmark stands where the two rivers meet

The fact-check — is the page true to the bible?

Once a fact is in the book, **Ctrl+B Shift+X** checks the **open paragraph** against it — and against your compiled world. It is looking for the moment your prose and your bible disagree: a journey too fast for its distance, a season that cannot follow the last, a claim that contradicts something you already fixed as true.

● ● ● Ctrl+B Shift+X — the paragraph, against the world

Fact-check · ch. 14 ¶ the-crossing

△ "they reached Rillmark by nightfall" — the bible puts Rillmark

six days' ride away; a single day is not plausible.

- no other conflicts with the Facts book.

When the facts disagree with each other

The subtler danger is not prose-versus-bible but **bible-versus-bible**: two claims you fixed as true, months apart, that cannot both hold. `inkhaven factcheck` reads the Facts book for these internal contradictions, and the SCHOLAR tools go further — `/contradict` surfaces the source-attributed clashes, `/relate` grades a new claim against what you already believe (does it **contradict**, **qualify**, **agree**, or say nothing).

FICTION

For fiction, the Facts book is your continuity bible — the invented truths the rest of the machinery in this book checks against. It is where “who is the elder sister” stops being something you hope you remembered.

NON-FICTION

For non-fiction, every kept fact remembers **where it came from**. The Research Assistant (its own companion book) gathers and cross-checks them; here they become the ground your argument — and every other check — stands on.

The Facts book is the floor of everything that follows. The knowledge graph draws its edges over these facts; SENTINEL’s continuity watch reads them; KEN asks who **knows** them. Get this floor solid, and the rest of the house has something to stand on.

WHAT YOU LEARNED

- The **Facts book** is your story bible made operational — the must-stay-true claims your manuscript can be checked against.
- Ctrl+B Shift+X fact-checks the open paragraph against the bible and the world; inkhaven factcheck finds contradictions **within** the bible.
- SCHOLAR's /contradict and /relate grade how a new claim stands against what you already hold true.

CHAPTER 2

2

The Knowledge Graph

A book is not a list; it is a **web**. Mara is Joren's sister and the war's orphan and the coronation's reluctant guest; the war connects to the treaty connects to the famine connects back to Mara. You hold these connections loosely in your head, and they fray. Inkhaven holds them exactly, in a **graph** — a map of how everything in your book relates to everything else — and lets you ask it questions.

TERM The graph

Inkhaven's **knowledge graph** is a layer of typed connections over the things in your book — paragraphs, facts, events, characters. An edge says **how** two things relate: this cites that, this event involves her, this claim contradicts that one. You never build it by hand; the tool derives it from what you have already written.

What the edges know

The graph is not decoration — every edge is one the tool can walk. A fact **contradicts** another; a paragraph **mentions** a character; a claim is **sourced from** a passage; an event **involves** the people who were there. That last edge is the one KEN will lean on two chapters from now: the graph already knows who was present when something happened.

● ● ● *inkhaven graph stats — the web, counted*

Graph	• 1,204 nodes	• 3,881 edges	
Mentions	 2,140	(paragraph → character / place)	
EventInvolves	... 612	(event → who was there)	
Contradicts	 18	(fact ≠ fact)	
Cites	 744	(claim → source)	
Declares	 167	(world.hjson → the fact it fixes)	

Ask your book a question

The graph would be a curiosity if you could only count it. What makes it an intelligence is that you can **talk to it**. `inkhaven graph ask` takes a plain question and **walks the graph** to answer it — searching, following neighbours, chasing contradictions and paths — and it is honest about what it does not record.

● ● ● *graph ask — a question, answered by traversal*

```
> inkhaven graph ask "who has reason to want the treaty to fail?"
```

```
Walking: search 'treaty' → neighbours → contradictions ...
- Sella loses her claim if the treaty holds (ch. 6, ch. 11).
- The northern lords are bound by it against their oath (ch. 9).
- I have no recorded motive for Joren — the graph is silent there.
```

Inside the editor the same power is the **Graph** AI scope (F9) and the graph hub (Ctrl+B z): ask a question and the answer is grounded in the passages **and** the edges touching them, with the evidence one keypress away.

The graph builds itself

You do not curate this web. Deep research proposes **contradicts** edges when two findings clash; **inkhaven graph link** suggests connections from a fact to its nearest kin; and an **edge inbox** (**graph pending**, or **i** in the hub) collects these proposals for you to keep or reject. The graph grows the way your book does — a little at a time, mostly on its own.

FICTION

For fiction, the graph is where “how does everyone connect to the murder” stops being a note card wall and becomes a question you can ask out loud.

NON-FICTION

For non-fiction, the graph is the shape of your argument — which claim rests on which source, and where two of your own findings quietly disagree.

WHAT YOU LEARNED

- The **knowledge graph** is a layer of typed edges over your book — mentions, event participants, contradictions, citations — derived from what you wrote.
- `inkhaven graph ask` (and the F9 Graph scope / `Ctrl+B z hub`) **walks** the graph to answer a plain question, grounded and honest about gaps.
- The graph **builds itself** — contradiction and link proposals collect in an edge inbox you keep or reject.

PART II

The Book Watches Itself

CHAPTER 3

3

Continuity — Where and When

Continuity errors are the ones readers write letters about. A character in two places on the same afternoon; a wound that heals and reopens; a sum of years that does not add up; a name that changes spelling between chapters. None of them are failures of imagination — they are failures of **bookkeeping**, and bookkeeping is exactly what a tool should do so you do not have to. Inkhaven's continuity intelligence is called SENTINEL: the book watching itself.

One ledger, every detector

SENTINEL unifies the deterministic continuity checks into a single ranked ledger. It watches four kinds of slip, plus one invariant nobody usually catches:

● ● ● *inkhaven continuity check — the unified ledger*

```
Continuity ledger · 3 findings
co_location (1)
  ⊗ Mara is in the tower and the courtyard within one scene
  (ch. 3)
numeric (1)
  △ "twelve years later" contradicts the dated events (ch. 8 →
  ch. 2)
introduce (1)
  ● "Cael" is named in ch. 2 before his first scene (ch. 5)
```

- **co-location** — a character in two places at once, without the travel to connect them.
- **timeline** — events that orphan or overlap where they cannot.
- **numeric** — a span of years, a distance, a count that reverses itself.
- **character-fact drift** — a detail about someone that quietly changes.
- **referenced-before-introduced** — an entity named before its first scene (the invariant that KEN, next chapter, extends from **existence** to **knowledge**).

The dashboard, and the jump

Ctrl+B Shift+I opens the ledger in the editor. Scroll it, and press **Enter** on any finding to jump straight to the paragraph where the slip happens — the whole point is to get you to the fix in one keystroke, not to hand you a report you then have to hunt through.

TERM Ambient watch

With `continuity.ambient` on, SENTINEL re-checks **only what your edit touched** on every save — the paragraph’s entities and its chapter — and shows the delta inline. It is the difference between a checkup you schedule and a nurse who never leaves the room.

The subtle cases, on request

The patterns above are deterministic and free. For the contradictions no pattern can see — two paragraphs that disagree in meaning, not in a number — press `k` in the ledger (or `inkhaven continuity check --coherence`) to run a cost-capped language pass over the book. Explicit, budgeted, and off by default: the free checks earn their keep first.

Continuity answers **where** and **when**. It is deliberately silent about one axis it was never built to see — what a character **knows**. That axis is the next chapter, and it is the reason this book exists.

WHAT YOU LEARNED

- **SENTINEL** unifies the continuity checks — co-location, timeline, numeric, character-fact drift, and referenced-before-introduced — into one ranked ledger.
- `Ctrl+B Shift+I` opens it; **Enter** jumps to the slip. `continuity.ambient` re-checks what each save touched.
- The core is free; the LLM **coherence pass** (`k / --coherence`) is explicit and cost-capped for the contradictions patterns can’t see.

CHAPTER 4

4

Who Knows What, When

The last chapter watched **where** your characters are. This one watches something no tool watched before it: **what they know**. It is the axis a mystery lives or dies on, and the most common invisible plot-hole in the whole craft — a character acting on something they have not learned yet. Bob mentions the murder in chapter four; he is not told of it until chapter six. A reader feels the wrongness and cannot place it. You, three hundred pages deep, cannot hold every character's knowledge in your head.

KEN can. Its name is an old word for the range of what a person knows — **beyond his ken** — and that is exactly what it tracks.

TERM KEN

KEN extends SENTINEL's referenced-before-**introduced** rule to referenced-before- **known**. Where continuity flags an entity named before it exists in the story, KEN flags a character acting on a fact before they could know it. Same forward walk, a new axis: not existence, but knowledge.

The grant — when could they know it?

KEN never **guesses** what a character knows. It derives the **grant** — the earliest moment they could know something — two deterministic ways. The first is free, from the timeline you already keep: anyone present at an event knows that event from the moment it happens. The second is a tag you place, telling KEN what matters:

● ● ● *The tags that declare knowledge*

```
secret:the betrayal          – this topic is a secret
know:the betrayal@Mara      – Mara learns it here
know:the betrayal          – the scene's pov: character
learns it
reveals:the heir's true name – this event lets that fact
slip
```

The use — and the break

With grants in hand, KEN walks the book forward. Wherever a character **references** a topic — speaks of it in attributed dialogue, or dwells on it in their own point-of-view scene — it asks the one question: **could they know this yet?** If the reference comes before the earliest grant, that is the break.

● ● ● *inkhaven knowledge — the epistemic check*

Knowledge check · 2 findings

- ⊗ `premature_knowledge` Bob speaks of "the murder" in ch. 4 – before learning it in ch. 6
- ⊗ `leaked_secret` Sella references "the heir's true name" in ch. 3 – never established to know it

Three kinds of break: **premature_knowledge** (a reference before the grant), **leaked_secret** (a `secret:` topic used by someone never told it), and **dropped_reveal** (a knowledge you set up and then never spent — the epistemic version of a gun on the mantel that never fires). KEN stays **silent** wherever it cannot ground a break; it never invents one.

Where it lives

`inkhaven knowledge` runs the check from the command line and exits non-zero `Ctrl+B` on a hard break, so you can gate a draft before it goes out. In the editor, `Shift+Z` opens the findings grouped by kind — **Enter** jumps to the paragraph where the character knew too much. And the same findings ride the revision worklist (`Ctrl+V Shift+R`), where you **decide** the fix: cut the reference, move the reveal earlier, or add the grant you forgot.

COST

The whole check is a forward walk plus name-matching — **no model, and it costs nothing at any book size**; it scales with your tags, not your page count, and does nothing at all until you place a tag. Only the opt-in `--deep` pass — for a character who acts knowing **without naming** the fact — asks a language model, and it is cost-capped like every other language pass in this book.

FICTION

For fiction this is the crown jewel — mystery, thriller, betrayal, any story with a secret. KEN is the reader's instinct given a memory: a secret stays secret until you choose to spend it.

NON-FICTION

For non-fiction the axis rarely bites — an argument has no secrets to keep — so KEN stays quiet, which is itself the right answer.

WHAT YOU LEARNED

- **KEN** tracks who knows what, when — the referenced-before-introduced invariant moved from existence to **knowledge**.
- The **grant** (when they could know it) comes from timeline presence + **secret:** / **know:** / **reveals:** tags; a **use** before the grant is the break.
- Findings: **premature_knowledge**, **leaked_secret**, **dropped_reveal** — surfaced in **inkhaven knowledge**, the **Ctrl+B Shift+Z** dashboard, and the **Ctrl+V Shift+R** worklist. Deterministic and free; **--deep** is the only, opt-in, model touch.

PART III

The Book Reads Itself

CHAPTER 5

5

The Read-Through

Every intelligence so far has read your book **small** — a paragraph, a break, a fact. But the reader who matters most does not read small. They read **forward, once, whole**, not knowing the ending, and their experience of the whole is the thing you most want to know and can least see, because you already know how it turns out. KEN's sibling, LECTOR, is that first reader — the book reading itself, end to end.

TERM LECTOR

LECTOR reads the manuscript the way a first-time reader does: forward, once, whole. It reports two things — the **shape** of the read (its dramatic rise and fall) and the **experience** of it (where a reader gets lost, bored, or set the book down).

Shape — measured from the prose

LECTOR measures each chapter's dramatic intensity **from the prose itself** — the density of dialogue, a stakes-and-conflict vocabulary, the acceleration of the sentences, the turn at a chapter's end — with no tagging required. Then it reads that realized curve against the shape your chosen framework intends, and marks the **saggy middle** where the story means to rise but the prose reads flat.

● ● ● *inkhaven readthrough — the shape of the read*

Read-through · 12 chapters · Hero's Journey

measured 
expected 

△ [shape_sag] the shape wants a rise around ch. 5 (~55%) but
the
prose reads flat (~12%).

Six frameworks are built in — including the four-movement, conflict-optional **kishōtenketsu** — and the right one is suggested from your declared genre unless you name it yourself.

Audience — what a first reader trips on

The other half walks the book forward carrying the **reader's** state — who they have met, what is still open, how the energy runs — and flags the problems a first reader actually hits, forward-only, so a later payoff never excuses an earlier dip:

● ● ● *The reader-experience findings*

confusion	an entity used before it's introduced
info_dump	too many new names in one chapter
attention_dip	a flat, eventless chapter
put_down_risk	a run of flat chapters – a likely place to stop
unpaid_setup	a promise raised but never paid off

A reader's eyes, on request

The forward walk cannot judge whether the stakes **land**. For that, `readthrough --deep` (or `k` in the `Ctrl+B Shift+A` dashboard) runs a single, cost-capped **synthetic first read** — a language model reacting chapter by chapter as a reader who does not know the ending, seeing only a recap of what came before. It is the one place the book gets a stranger's opinion, and it is always something you ask for.

LECTOR does not replace your first reader. It catches what you would wince at before you hand them the pages.

WHAT YOU LEARNED

- **LECTOR** reads the book forward, once, whole — reporting its **shape** (intensity vs. the framework's intended curve) and the **audience** experience.
- Shape is measured from the prose (no tagging); audience flags confusion, info-dump, attention-dip, put-down risk, and unpaid setup, forward-only.
- `inkhaven readthrough` and `Ctrl+B Shift+A`; the cost-capped **synthetic first read** (`--deep / k`) is the only model touch.

CHAPTER 6

6

The Voices

Give three characters the same lines and a weak book is exposed: they all sound like the author. A strong book gives each a voice you could recognise with the dialogue tags stripped away — a rhythm, a vocabulary, a way of hedging or not. Knowing whether your cast actually sounds like different people is hard from the inside; you hear the voice you **meant**, not the one on the page. CHORUS listens for you.

TERM CHORUS

CHORUS profiles the voices of your book at the scale of the whole cast. It fingerprints each character's speech, measures how **distinct** they are from one another, and watches the discipline of point of view and register — synthesised by a reader in the inner family, the **Inner Stylist**.

A fingerprint, and a distinctiveness matrix

CHORUS builds a statistical fingerprint of each character's dialogue — sentence rhythm, vocabulary richness, how often they ask, exclaim, hedge — and then z-scores the whole cast against itself to find the voices that read **alike**.

● ● ● *inkhaven chorus — the voices, compared*

Voice distinctiveness · 5 speakers
 most distinct Grandmother (archaic, long clauses)
 least distinct Joren ↔ Cael (0.31) — they read the same
 ▲ two flat voices: consider sharpening Cael's diction.

Sparse voices — a character with three lines — are profiled but never flagged; there is not enough to judge, and CHORUS says so rather than inventing a verdict.

Whose head, and does the tense hold

The other pillar is **discipline**. Declare a scene's point of view with a **pov:** tag, and CHORUS flags a **head-hop** — a named non-viewpoint character whose interiority leaks onto the page — and, in English, a **tense slip**.

HONEST ABOUT RUSSIAN

Tense detection is **English-only, by design**. Russian tense is aspect, and the historical present is legitimate prose; nothing here models aspect, so CHORUS says so rather than false-flag it. Character voice and head-hop work in every supported language.

The Inner Stylist

All of this — voice, distinctiveness, drift, point of view, register — is gathered by the **Inner Stylist**, the seventh reader in Inkhaven’s inner family, into warm observations you can act on (`Ctrl+B J → Y`, or the `Ctrl+B Shift+C` review pass). It coaches; it never rewrites. A voice is yours to give; CHORUS only tells you whether you gave it.

FICTION

For fiction this is the difference between a cast and a crowd — the assurance that the reader can tell who is speaking with the tags removed.

NON-FICTION

For non-fiction voice matters less, but **register drift** — a chapter that slides from plain into jargon and back — is exactly the kind of unevenness CHORUS catches.

WHAT YOU LEARNED

- **CHORUS** profiles the whole cast’s voices: a per-character fingerprint plus a **distinctiveness matrix** that flags voices reading alike.
- It watches **discipline** too — head-hops against a `pov:` tag, and English tense slips (Russian excluded by design).
- The **Inner Stylist** synthesises it into coaching observations; it never rewrites.

PART IV

Knowing You're Getting Somewhere

CHAPTER 7

7

Did It Get Better?

Every intelligence in this book diagnoses the draft **in front of you**. But revision is a leap of faith unless you can see it working, and the honest question at the end of a hard week — **did I actually make it better, or did I just move the furniture?** — has no answer if nothing remembers what the book measured last time. CHRONICLE remembers. It is the book's memory of its own drafts.

TERM Milestone

A **milestone** is a snapshot of every reader's verdict at one moment — taken with **inkhaven chronicle** mark "draft-2". It captures the whole diagnostic state (every finding, tallied and fingerprinted) so a later draft can be measured against it.

Mark, then trend

Stamp a milestone before a serious pass. Afterward, **inkhaven chronicle** captures the book again and **trends** it against that mark — every count read the honest way, where **fewer is better**, and regressions sorted to the top so collateral damage cannot hide:

● ● ● *inkhaven chronicle — since the last milestone*

```
Chronicle — since "draft-2" (2026-08-03) → now
findings          31 → 27 ▼ 4 fewer
errors            4 → 2 ▼ cleared 2
shape sag         3 → 1 ▼ improved
confusion (ch.7)  0 → 1 ▲ NEW — introduced by your
edits
```

Cleared, and introduced

The line that matters most is the split. Because every finding carries a stable identity, **CHRONICLE** knows exactly which ones your revision **cleared** and which **new** ones it **introduced** — proof the work landed, and an early warning on the problem you created reaching for the fix:

● ● ● *The revision, weighed*

✓ 6 cleared ▲ 1 introduced · 20 unchanged

introduced (new since the last mark):

▲ confusion ch. 7 an entity used before it's
introduced

Press **Enter** on an introduced finding in the **Ctrl+B Shift+U** dashboard and you jump straight to the paragraph your last edits broke. Good revision often trades one problem for another; this is how you catch the trade before a reader does.

PURE MEASUREMENT

CHRONICLE has **no prose-write path anywhere**. It only ever measures. And because it reads findings the other intelligences already computed, it costs nothing to keep a running record of your book's whole health, draft to draft.

CHRONICLE closes the loop the rest of this book opens: the readers diagnose, you revise, and CHRONICLE tells you whether it worked.

WHAT YOU LEARNED

- A **milestone** (chronicle mark) snapshots every reader's verdict; inkhaven chronicle trends the live book against it, every count **fewer-is-better**.
- The signature move is **cleared vs. introduced** — which findings your revision resolved, and which new ones it created.
- **Ctrl+B Shift+U** opens the dashboard; **Enter** jumps to an introduced finding. Pure measurement — it never edits, and it costs nothing.

PART V

All Together

CHAPTER 8

8

A Scene Through Every Check

Read separately, these intelligences are seven tools. Used together, on one scene, they are a single sense — the feeling of **knowing your book** that no writer has unaided. Let us take one chapter through all of them, in the order you would actually reach for them, and watch the book come into focus.

The scene: chapter seven, in which Mara confronts Sella about the betrayal.

First, is it true to the world?

Before anything else, the facts. **Ctrl+B Shift+X** checks the page against the Facts book and the compiled world.

● ● ● 1 · *fact-check*

Fact-check · ch. 7

△ "she rode from Rillmark that morning" – the bible puts Rillmark six days away. Fix the geography or the timing.

Then, does the continuity hold?

Ctrl+B Shift+I – the continuity ledger. Physical and temporal first.

● ● ● 2 · *continuity (SENTINEL)*

⊗ *co_location* Mara is at Rillmark and the capital in one scene (ch. 7)

Could she know what she confronts Sella with?

The question continuity cannot ask. **Ctrl+B Shift+Z** – KEN.

● ● ● 3 · *knowledge (KEN)*

⊗ *premature_knowledge* Mara speaks of "the betrayal" in ch. 7 –
she is not told of it until ch. 9

There it is — the invisible break. Mara accuses Sella of a thing she has not learned. Move the reveal earlier, or give Mara a grant, and the scene holds.

Do they sound like two people, and how does it read?

Ctrl+B J → Y — the voices. And the whole book's shape, Ctrl+B Shift+A.

● ● ● 4 · voice (CHORUS) + read-through (LECTOR)

△ Mara ↔ Sella (0.29) — the confrontation reads in one voice.

● [attention_dip] ch. 7 runs low-energy after a flat ch. 6.

Act on it — and confirm it landed

Every finding above rides one worklist, Ctrl+V Shift+R, where each gets the right kind of help: a rewrite you review as a diff, a decision you make (“move the reveal or cut the line?”), a brief where structure is yours to move. Fix the scene there — and then, before the next pass, mark a milestone and let CHRONICLE tell you the truth:

● ● ● 5 · did it get better? (CHRONICLE)

Chronicle — since "before ch.7 pass" → now

✓ 3 cleared (co_location, premature_knowledge, the voice flag)

▲ 0 introduced

Three real problems, invisible an hour ago, found and cleared with nothing broken in their place. That is what it means to know your book — not to hold it all in your head, which no one can, but to have an instrument for every question you would otherwise have to answer from memory, and the honesty to tell you when the answer changed.

WHAT YOU LEARNED

- The intelligences compose: facts → continuity → knowledge → voice → shape → act → measure, on one scene.
- Everything converges on the `Ctrl+V Shift+R` worklist to act, and `Ctrl+B Shift+U` (CHRONICLE) to confirm the fix landed without collateral damage.
- Knowing your book is not holding it in your head — it is having an instrument for every question, and the honesty of a changed answer.

APPENDIX A



The Knowledge Commands

Every intelligence in this book has a command line and, where it lives in the editor, a chord. This is the whole set on two pages, for when you know what you want and just need the way to say it.

From the command line

inkhaven
factcheck

Contradictions within the Facts book; SCHOLAR /contradict, /relate.

inkhaven graph ask "..."	Answer a plain question by walking the knowledge graph.
inkhaven graph stats	Node + edge counts, per kind. rebuild / neighbors / paths / contradicting.
inkhaven continuity check	The SENTINEL ledger. --only / --skip / --json; --coherence for the LLM pass.
inkhaven knowledge	The KEN epistemic check. --json; --deep for implied_irony. Non-zero exit on a break.
inkhaven readthrough	The LECTOR read-through. --deep for the synthetic first read; --json.
inkhaven chorus	Voice profiles + the distinctiveness matrix. scan / report / stylist.
inkhaven chronicle	The draft trend since the last mark. mark / list / diff / --json.

Inside the editor

Ctrl+B Shift+X	Fact-check the open paragraph against the world.
Ctrl+B z	The graph hub — neighbourhood, the edge inbox, graph ask (F9 = the Graph scope).
Ctrl+B Shift+I	The SENTINEL continuity ledger (k = the LLM coherence pass).
Ctrl+B Shift+Z	The KEN knowledge dashboard — who knows what, when.
Ctrl+B Shift+A	The LECTOR read-through dashboard (k = the synthetic first read).
Ctrl+B J → Y	The Inner Stylist — CHORUS's voice observations.
Ctrl+B Shift+U	The CHRONICLE draft-history dashboard (m = mark this draft).
Ctrl+V Shift+R	The Editorial Pass — every reader's findings, one worklist, acted on.
Ctrl+B Shift+C	The unified review pass — the fast checks in one sweep.

The tags that declare knowledge (KEN)

- secret:<topic>** Marks <topic> a secret — a reference by someone ungranted is a leak.
- know:<topic>** Grants the scene's pov: character knowledge of <topic> here.
- know:<topic>@<name>** Grants <name> the knowledge explicitly.
- reveals:<topic>** On an event's paragraph — binds the event to the fact it reveals.
- pov:<name>** Declares a scene's viewpoint — used by CHORUS and KEN alike.

From a script (Bund)

Every check reads out to Bund, read-only, for hooks and gates: `ink.knowledge.{grants, findings, check}`, `ink.continuity.{findings, check}`, `ink.readthrough.{report, curve, check}`, `ink.chronicle.{marks, trend, check}`, and the graph words. Each `check` returns a `clean` flag — a one-line pass/fail gate for a pre-submit script.

APPENDIX B

B

Glossary

The vocabulary of knowing your book, gathered in one place.

TERM **Advisory**

The governing principle: every intelligence observes and reports, and never edits your prose on its own. A change is always a diff you confirm, with your old words a keystroke back.

TERM Fact

A specific, checkable claim your book rests on — a date, a name, a distance — kept in the Facts system book, apart from your prose, so the tool can hold it steady.

TERM Knowledge graph

A layer of typed connections over your book — mentions, event participants, contradictions, citations — derived from what you wrote, walkable, and answerable with **graph ask**.

TERM Continuity (SENTINEL)

The book watching **where** and **when**: co-location, timeline, numeric, character-fact drift, and referenced-before-introduced, in one ranked ledger.

TERM Grant

In KEN, the earliest moment a character could know a topic — derived from timeline presence (they were there) or a **know:** / **reveals:** tag you place.

TERM Epistemic break

A character using knowledge before they have it: **premature_knowledge**, a **leaked_secret**, or a **dropped_reveal** (knowledge set up and never spent). The axis KEN watches that continuity cannot.

TERM Read-through (LECTOR)

The book read forward, once, whole, as a first reader — its **shape** (measured intensity vs. the framework's curve) and its **audience** experience (confusion, info-dump, attention-dip, put-down risk, unpaid setup).

TERM Distinctiveness matrix

CHORUS's z-scored comparison of the whole cast's voice fingerprints — the thing that flags two characters who read alike.

TERM Milestone

A snapshot of every reader's verdict at one moment (**chronicle mark**), so a later draft can be trended against it — cleared vs. introduced.

TERM Deterministic / cost-capped

Deterministic means the tool reads your manuscript and its structure with no model, at no cost. Where a language model genuinely helps, it is **cost-capped** and opt-in — you ask, you see the estimate, the default is off.

TERM The worklist

The Editorial Pass (**Ctrl+V Shift+R**): every reader's findings unified into one ranked list, each acted on with the right kind of help — a reviewed rewrite, a decision you make, or a brief where the structure is yours to move.

AFTERWORD

About the author



Vladimir Ulogov.

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. Early in his career he worked on monitoring and telemetry platforms; later years took him into federated observability, telemetry buses, and the architecture of systems that have to make sense of millions of data points without losing the thread.

Observability, in the end, is the art of **holding a system too large for one head** — of watching it without losing the thread, and never reporting a thing it cannot back up. It is not an accident that a tool he built for writers carries the same instinct to the desk: a book, past a certain size, is exactly such a system, and it too

deserves an instrument that holds it for you.

What makes him slightly unusual in his corner of the industry is a tendency to write his own tools — not in the sense of small utilities, but in the sense of programming languages. The Bund language (its compiler, its VM, its document store, its parser) lives in a long series of Rust crates on crates.io. `rust_dynamic`, `rust_multistack`, `rust_multistackvm`, `bundcore`, `zbus_universal_data_gateway` — each is a building block that exists because the off-the-shelf options didn't fit the shape of the work.

Why these intelligences exist

Inkhaven is Vladimir's personal reflection on how a literary tool can help the people who write books. The intelligences in this book are one answer to a specific frustration: that a manuscript, past a certain length, outgrows the mind writing it — and the tools writers use offer no help holding it. They format the words; they do not **understand** them.

A novelist loses track of who knows what, whether the continuity holds, whether two characters have started to sound alike — not from carelessness, but because no one holds a whole book at once. A non-fiction author loses the thread between a claim and its source, and never sees the two of their own findings that quietly disagree. These features were built to close that gap: to let the tool that holds your words also **know** them — watch the facts, the continuity, the knowledge, the shape, and the voices — so the writer is free to do the one thing no tool can, which is write.

A work of love

Inkhaven is open source. The licence is permissive — you can read it, fork it, study it, modify it, and pass it on. Strictly speaking, the licence also lets you sell it. The author would, gently but firmly, disagree with you doing that. Inkhaven was not designed as a *for sale* project. It is a work of love made for the authors who can least afford to pay for software, and turning it into a commercial product would betray the reason it exists. So please — don't.

It carries no analytics, no telemetry, no upsell. The binary will never phone home; the project will never have an “Enterprise” tier you have to escape from.

This was a deliberate choice. There are excellent commercial tools for writing. They cost money — which is fine for many writers, and a barrier for many more. Inkhaven exists for the second group: for the graduate student writing a dissertation on a battered laptop, for the novelist who shouldn't have to pick between rent and software, for the engineer drafting in the same terminal where they already write code, for anyone who would benefit from a tool that respects their work without asking for a credit-card number.

A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without permission, by people who will never meet.

If Inkhaven helps you finish your book — that is enough. If it gives you a chord pattern you adapt into your own tool — that's a gift back to the larger project of making software writers can love. As a society, we achieve the greatest things when we help each other rather than compete with each other.

Where to find more

GitHub	@vulogov — the source for Inkhaven, Bund, and the dozen-plus Rust crates that carry the infrastructure. Issues and PRs welcome.
LinkedIn	/in/vladimirulogov — posts on observability, the occasional long-form essay.
YouTube	@vulogov — talks and walkthroughs from the conference trail.
X / Twitter	@vladimir_ulogov

If a fact you grounded ever surprises you, or a command trips you up and you can't find the answer in this book — open an issue on GitHub. The author reads them.

KNOW YOUR BOOK

END OF THE BOOK · INKHAVEN 2.6.0